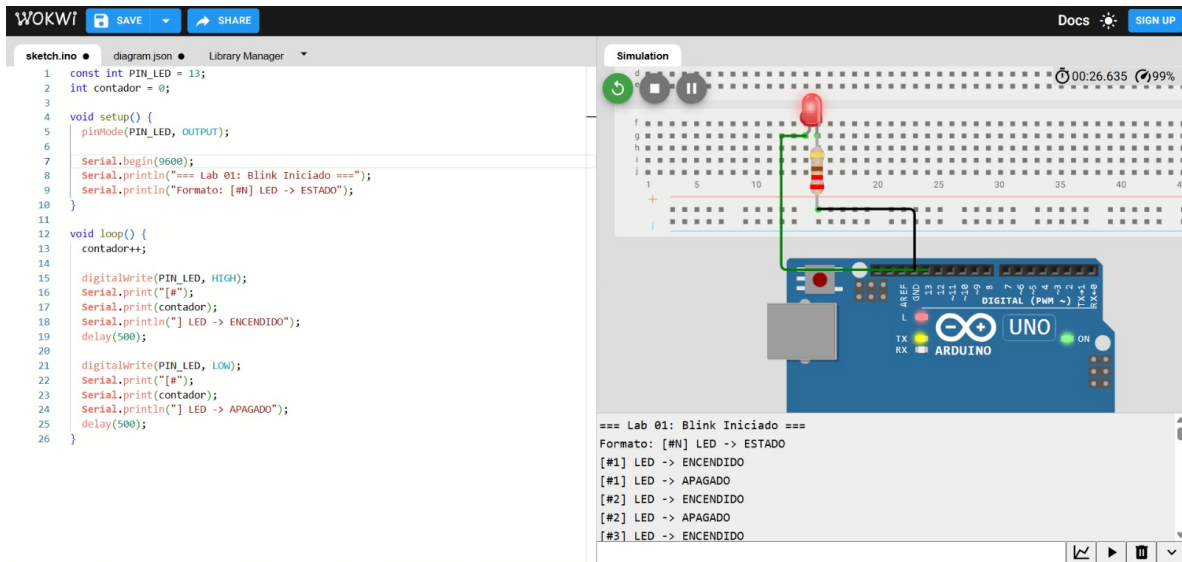


Laboratorio 1



Programa que indica los estados “ACTIVO” e “INACTIVO”.

Se utilizó el código base del manual de prácticas como referencia inicial para realizar modificaciones posteriores que se muestran en las siguientes imágenes del documento.

Ejercicio 1

The screenshot displays the Wokwi IDE interface. On the left, the 'sketch.ino' file contains the following code:

```
1 const int PIN_LED = 13;
2 int contador = 0;
3
4 void setup() {
5   pinMode(PIN_LED, OUTPUT);
6   Serial.begin(9600);
7   Serial.println("== Lab 01: Blink Iniciado ==");
8   Serial.println("Formato: [#N] LED -> ESTADO");
9 }
10
11 void loop() {
12   contador++;
13
14   digitalWrite(PIN_LED, HIGH);
15   Serial.print("#");
16   Serial.print(contador);
17   Serial.println("] LED -> ENCENDIDO");
18   delay(200); // Encendido 200 ms
19
20   digitalWrite(PIN_LED, LOW);
21   Serial.print("#");
22   Serial.print(contador);
23   Serial.println("] LED -> APAGADO");
24   delay(800); // Apagado 800 ms
25 }
26
```

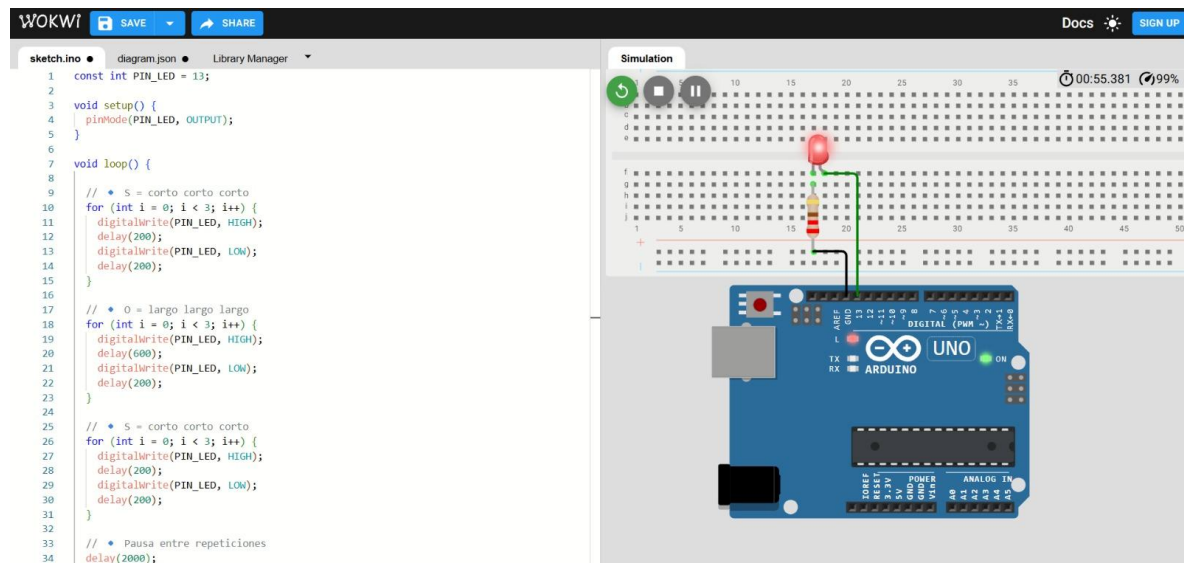
The center panel shows a breadboard circuit with an Arduino Uno connected to a red LED. The LED's anode is connected to digital pin 13, and its cathode is connected to ground. A 220Ω resistor is placed in series with the LED.

The right panel shows the 'Simulation' window with a timer at 00:32.279 and 100% zoom. Below the simulation area, the serial monitor displays the following output:

```
=== Lab 01: Blink Iniciado ===
Formato: [#N] LED -> ESTADO
[#1] LED -> ENCENDIDO
[#1] LED -> APAGADO
[#2] LED -> ENCENDIDO
[#2] LED -> APAGADO
[#3] LED -> ENCENDIDO
```

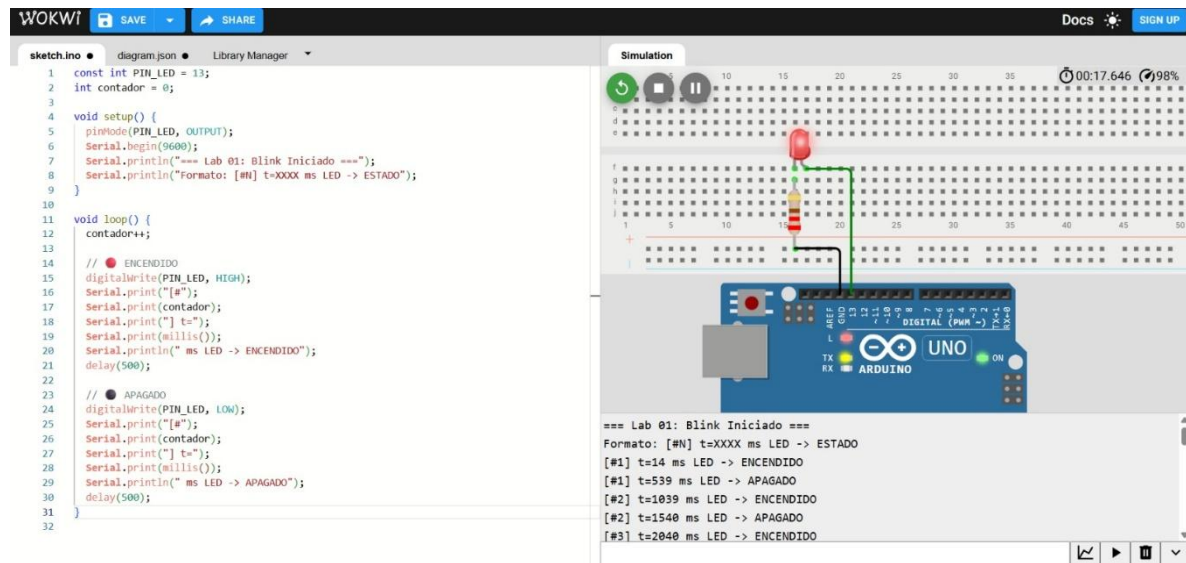
En este ejercicio se ajustaron los tiempos de retardo (delay) del LED, estableciendo 200 ms para el encendido y 800 ms para el apagado, con el objetivo de observar cómo cambia el ritmo de parpadeo y la forma en que se muestran los mensajes en el monitor serial indicando los estados de encendido y apagado.

Ejercicio 2



En este ejercicio se modificó el código para generar distintos tiempos de encendido y apagado del LED, permitiendo representar señales cortas y largas que simulan el código Morse (SOS), utilizando ciclos repetitivos para controlar la duración de cada pulso y así visualizar el patrón de parpadeo.

Ejercicio 3

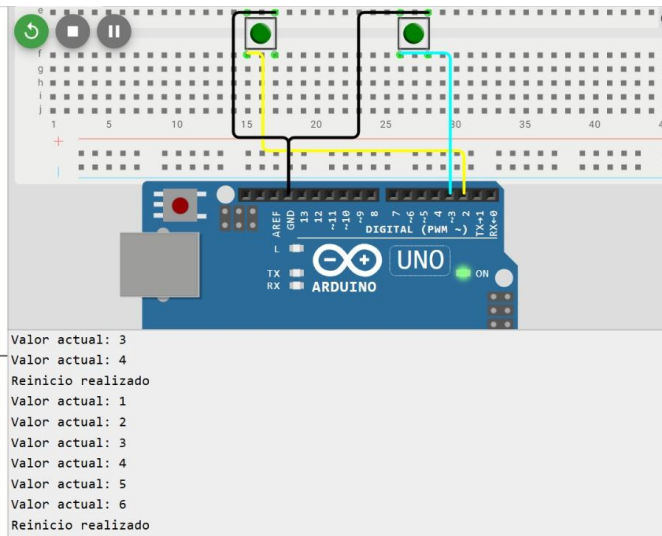


En esta última parte de la práctica se modificó el código para mostrar en el monitor serial el tiempo transcurrido en milisegundos desde que se ejecuta el programa, junto con el estado del LED (encendido y apagado), permitiendo observar cómo evoluciona el tiempo durante la ejecución continua del sistema.

Laboratorio 2

Ejercicio1

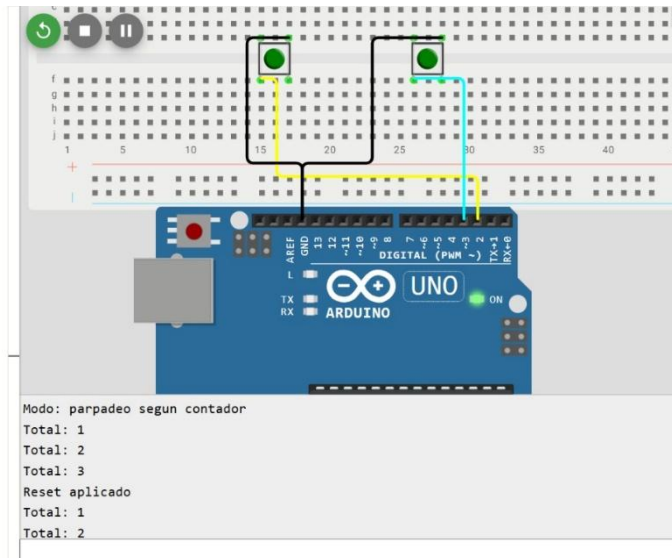
```
1  const int BTN_CONT = 2;
2  const int BTN_RESET = 3;
3  const int LED = 13;
4
5  int valor = 0;
6
7  bool estadoPrevCont = HIGH;
8  bool estadoPrevReset = HIGH;
9
10 void setup() {
11   pinMode(BTN_CONT, INPUT_PULLUP);
12   pinMode(BTN_RESET, INPUT_PULLUP);
13   pinMode(LED, OUTPUT);
14
15   Serial.begin(9600);
16   Serial.println("Inicio del sistema - contador = 0");
17 }
18
19 void loop() {
20
21   bool estadoCont = digitalRead(BTN_CONT);
22   bool estadoReset = digitalRead(BTN_RESET);
23
24   // --- BOTÓN CONTADOR ---
25   if (!estadoCont && estadoPrevCont) {
26     delay(50);
27     valor++;
28
29     Serial.print("Valor actual: ");
30     Serial.println(valor);
31
32     // Feedback LED
33     digitalWrite(LED, HIGH);
34 }
```



El código implementa un sistema de conteo con dos botones usando resistencias internas pull-up, donde uno incrementa un contador y el otro lo reinicia, mostrando el valor en el monitor serial y utilizando un LED como retroalimentación visual, además de emplear detección de flanco y anti-rebote para evitar lecturas erróneas.

Ejercicio 2

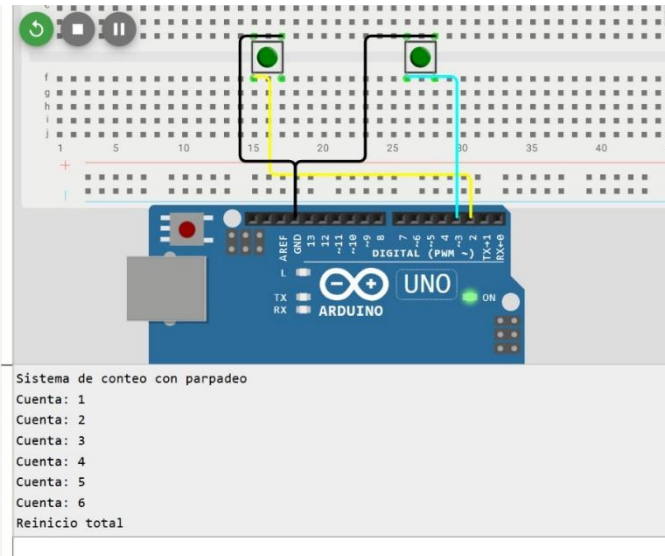
```
1  const int BTN_SUMA = 2;
2  const int BTN_CLEAR = 3;
3  const int LED_PIN = 13;
4
5  int conteo = 0;
6
7  bool prevSuma = HIGH;
8  bool prevClear = HIGH;
9
10 void setup() {
11   pinMode(BTN_SUMA, INPUT_PULLUP);
12   pinMode(BTN_CLEAR, INPUT_PULLUP);
13   pinMode(LED_PIN, OUTPUT);
14
15   Serial.begin(9600);
16   Serial.println("Modo: parpadeo segun contador");
17 }
18
19 void loop() {
20
21   bool estadoSuma = digitalRead(BTN_SUMA);
22   bool estadoClear = digitalRead(BTN_CLEAR);
23
24   // ---- BOTÓN SUMA ----
25   if (!estadoSuma && prevSuma) {
26     delay(50);
27
28     conteo++;
29
30     Serial.print("Total: ");
31     Serial.println(conteo);
32
33     // Parpadeo proporcional
34     for (int k = 0; k < conteo; k++) {
```



Este código implementa un sistema de conteo con dos botones usando resistencias pull-up internas, donde uno incrementa el contador y provoca que un LED parpadee tantas veces como el valor acumulado, mientras que el otro botón reinicia el contador a cero mostrando el cambio en el monitor serial y utilizando un parpadeo largo del LED como confirmación, además de incluir detección de flanco y anti-rebote para evitar errores en la lectura.

Ejercicio 3

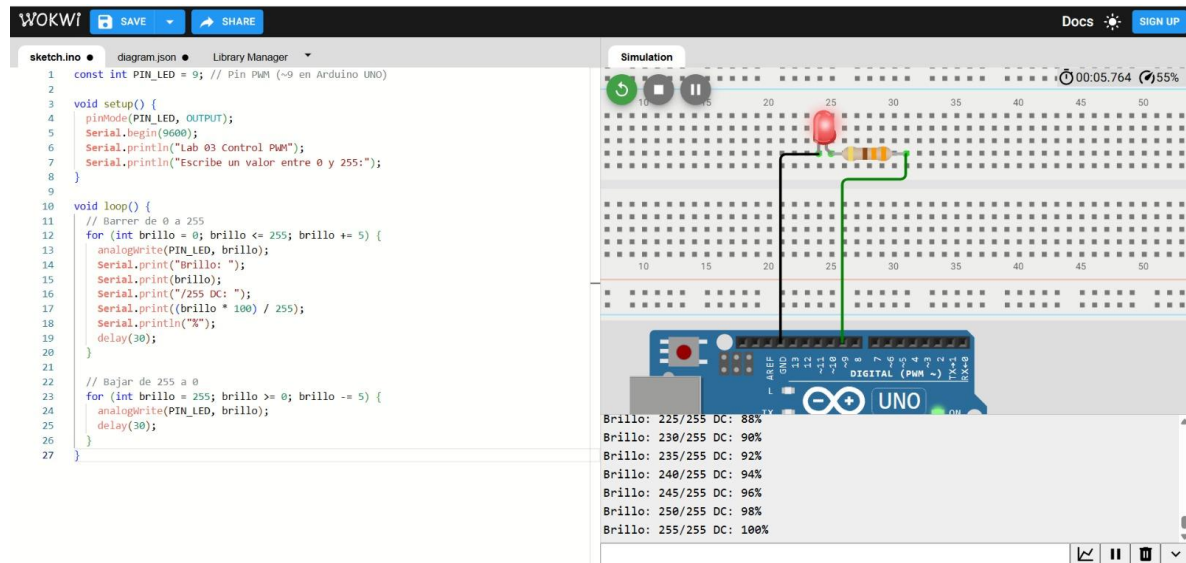
```
1  const int BTN_UP = 2;  
2  const int BTN_RST = 3;  
3  const int LED_OUT = 13;  
4  
5  int valorActual = 0;  
6  
7  bool lastUp = HIGH;  
8  bool lastRst = HIGH;  
9  
10 void setup() {  
11   pinMode(BTN_UP, INPUT_PULLUP);  
12   pinMode(BTN_RST, INPUT_PULLUP);  
13   pinMode(LED_OUT, OUTPUT);  
14  
15   Serial.begin(9600);  
16   Serial.println("Sistema de conteo con parpadeo");  
17 }  
18  
19 void loop() {  
20  
21   bool lecturaUp = digitalRead(BTN_UP);  
22   bool lecturaRst = digitalRead(BTN_RST);  
23  
24   // ----- SUMAR -----  
25   if (lecturaUp == LOW && lastUp == HIGH) {  
26     delay(50);  
27  
28     valorActual += 1;  
29  
30     Serial.print("Cuenta: ");  
31     Serial.println(valorActual);  
32  
33     ejecutarParpadeo(valorActual);  
34   }
```



Este código implementa un sistema de conteo con dos botones utilizando resistencias pull-up internas, donde uno incrementa el valor almacenado y activa una función que hace parpadear un LED tantas veces como el número actual, mientras que el otro botón reinicia el contador a cero mostrando el resultado en el monitor serial y utilizando una señal visual, incorporando además detección de flanco y anti-rebote para garantizar lecturas correctas.

Laboratorio 3

Ejercicio1



En este ejercicio el brillo del LED inicia en 0 y se controla mediante un ciclo for que incrementa su valor de 5 en 5 hasta llegar al máximo (255), aumentando progresivamente la intensidad mediante PWM, y posteriormente otro ciclo reduce el brillo de la misma forma, logrando un efecto de encendido y apagado gradual del LED.

Ejercicio 2

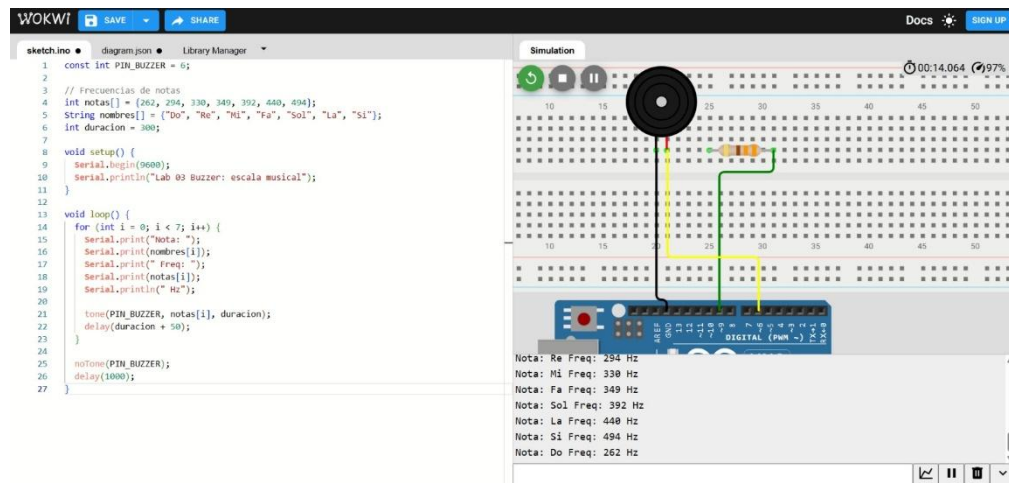
The screenshot displays the WOKWI simulation interface. On the left, the code editor shows the following C++ code:

```
1 #include <math.h>
2
3 const int PIN_LED = 9;
4 float angulo = 0.0;
5
6 void setup() {
7   pinMode(PIN_LED, OUTPUT);
8 }
9
10 void loop() {
11   // Convertimos seno (-1 a 1) a (0 a 1)
12   float seno = (sin(angulo) + 1.0) / 2.0;
13
14   // Escalamos a 0-255
15   int brillo = (int)(seno * 255);
16
17   analogWrite(PIN_LED, brillo);
18
19   angulo += 0.05; // Velocidad
20
21   if (angulo > 2 * PI) angulo = 0;
22
23   delay(15);
24 }
```

On the right, the simulation shows an Arduino Uno board connected to a breadboard. A red LED is connected to digital pin 9 (labeled 'DIGITAL (PWM ~)') and ground. The simulation status bar at the top right indicates a runtime of 00:14.194 and 59% completion.

En este ejercicio se utiliza una función matemática seno ($\sin$) para generar una variación suave en el brillo del LED, donde un ángulo se incrementa continuamente y su valor es transformado de un rango de -1 a 1 hacia 0 a 1, luego se escala de 0 a 255 para controlar el PWM, logrando un efecto de encendido y apagado progresivo, reiniciando el ciclo al completar una vuelta completa.

Ejercicio 3



En este ejercicio se utilizan arreglos para almacenar las frecuencias de las notas musicales y sus respectivos nombres, junto con una variable que define la duración de cada nota en milisegundos; el buzzer reproduce cada frecuencia mediante la función `tone()` recorriendo el arreglo, generando una escala musical, y al finalizar se realiza una pausa antes de repetir la secuencia.